UNIVERSIDAD DE GRANADA

UNIVERSIDAD DE GRANADA

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación (ETSIIT)

Especificación formal del protocolo en ABNF

Autor:
Adrián Macías Caballero
Paula Genaro Soria

Asignatura:
Desarrollo de Aplicaciones en Red

18 de marzo de 2026

Índice

1. Especificación formal del protocolo en ABNF **2**
1.1. Gramática ABNF del protocolo 2
1.2. Descripción general de la gramática 5
1.2.1. Peticiones 5
1.2.2. Respuestas 6
1.2.3. Notificaciones 6
1.3. Elementos internos del protocolo 6
1.4. Campos y valores literales 7
1.5. Tipos básicos 7
1.6. Observación final 7

1. Especificación formal del protocolo en ABNF

En este apartado se define formalmente el formato de los mensajes intercambiados en el protocolo mediante **ABNF** (*Augmented Backus-Naur Form*). Esta especificación describe la estructura sintáctica de las peticiones, respuestas y notificaciones que circulan entre cliente y servidor.

El protocolo utiliza **serialización en JSON codificada en UTF-8**. Además, cuando se transporta sobre **TCP**, cada mensaje finaliza con un salto de línea **LF** (`%x0A`), lo que permite delimitar claramente cada unidad de información transmitida.

> **Nota:** aunque el formato JSON no exige un orden fijo de los campos, en esta especificación se adopta un **orden canónico recomendado** con el objetivo de facilitar la interoperabilidad entre implementaciones.

1.1. Gramática ABNF del protocolo

```
; ==========================================================
; PROTOCOLO METEO
; Serializacin JSON UTF-8
; En TCP, cada mensaje termina en LF (%x0A)
; ==========================================================

tcp-message     = message LF
LF              = %x0A
OWS             = *(SP / HTAB)

message         = request / response / notification

; =========================
; PETICIONES
; =========================

request         = list-req / get-req / sub-req / unsub-req

list-req        = "{" OWS q-command OWS ":" OWS q-LIST OWS "}"

get-req         = "{" OWS
                  q-command OWS ":" OWS q-GET
                  [ OWS "," OWS q-city OWS ":" OWS city ]
                  OWS "}"

sub-req         = "{" OWS
                  q-command OWS ":" OWS q-SUB
                  [ OWS "," OWS q-city OWS ":" OWS city ]
                  [ OWS "," OWS q-variables OWS ":" OWS variable-list ]
                  OWS "}"
```

```
unsub-req       = "{" OWS
                  q-command OWS ":" OWS q-UNSUB
                  [ OWS "," OWS q-city OWS ":" OWS city ]
                  OWS "}"

; =========================
; RESPUESTAS
; =========================

response        = list-resp / get-resp-ok / sub-resp-ok /
                  unsub-resp-ok / error-resp / proxy-error-resp

list-resp       = "{" OWS
                  q-status OWS ":" OWS status-200 OWS "," OWS
                  q-type   OWS ":" OWS q-RESP_LIST OWS "," OWS
                  q-data   OWS ":" OWS city-list OWS "," OWS
                  q-msg    OWS ":" OWS text
                  OWS "}"

get-resp-ok     = "{" OWS
                  q-status OWS ":" OWS status-200 OWS "," OWS
                  q-city   OWS ":" OWS city OWS "," OWS
                  q-data   OWS ":" OWS weather-data
                  OWS "}"

sub-resp-ok     = "{" OWS
                  q-status  OWS ":" OWS status-200 OWS "," OWS
                  q-msg     OWS ":" OWS text OWS "," OWS
                  q-current OWS ":" OWS weather-data
                  OWS "}"

unsub-resp-ok   = "{" OWS
                  q-status OWS ":" OWS status-200 OWS "," OWS
                  q-msg    OWS ":" OWS text
                  OWS "}"

error-resp      = "{" OWS
                  q-status OWS ":" OWS (status-400 / status-404 / status
    -500) OWS "," OWS
                  q-msg    OWS ":" OWS text
                  OWS "}"

proxy-error-resp = error-resp

; =========================
; NOTIFICACIONES
; =========================

notification    = "{" OWS
```

```
                  q-type OWS ":" OWS q-NOTIF OWS "," OWS
                  q-city OWS ":" OWS city OWS "," OWS
                  q-data OWS ":" OWS notif-data
                  OWS "}"

; =========================
; ESTRUCTURAS INTERNAS
; =========================

city-list       = "[" OWS city *( OWS "," OWS city ) OWS "]"

variable-list   = "[" OWS variable *( OWS "," OWS variable ) OWS "]"

variable        = q-temp / q-hum / q-pres / q-wind

weather-data    = "{" OWS
                  temp-member OWS "," OWS
                  hum-member  OWS "," OWS
                  pres-member OWS "," OWS
                  wind-member
                  OWS "}"

notif-data      = "{" OWS
                  change-member *( OWS "," OWS change-member )
                  OWS "}"

change-member   = temp-member / hum-member / pres-member / wind-member

temp-member     = q-temp OWS ":" OWS number
hum-member      = q-hum  OWS ":" OWS number
pres-member     = q-pres OWS ":" OWS number
wind-member     = q-wind OWS ":" OWS number

; =========================
; NOMBRES DE CAMPOS
; =========================

q-command       = DQUOTE "command" DQUOTE
q-city          = DQUOTE "city" DQUOTE
q-variables     = DQUOTE "variables" DQUOTE
q-status        = DQUOTE "status" DQUOTE
q-type          = DQUOTE "type" DQUOTE
q-data          = DQUOTE "data" DQUOTE
q-msg           = DQUOTE "msg" DQUOTE
q-current       = DQUOTE "current" DQUOTE

; =========================
; VALORES LITERALES
; =========================
```

```
q-LIST          = DQUOTE "LIST" DQUOTE
q-GET           = DQUOTE "GET" DQUOTE
q-SUB           = DQUOTE "SUB" DQUOTE
q-UNSUB         = DQUOTE "UNSUB" DQUOTE
q-RESP_LIST     = DQUOTE "RESP_LIST" DQUOTE
q-NOTIF         = DQUOTE "NOTIF" DQUOTE

q-temp          = DQUOTE "temp" DQUOTE
q-hum           = DQUOTE "hum" DQUOTE
q-pres          = DQUOTE "pres" DQUOTE
q-wind          = DQUOTE "wind" DQUOTE

status-200      = "200"
status-400      = "400"
status-404      = "404"
status-500      = "500"

; =========================
; TIPOS BSICOS
; =========================

city            = string
text            = string

number          = ["-"] int [ frac ]
int             = "0" / (NZDIGIT *DIGIT)
NZDIGIT         = %x31-39
frac            = "." 1*DIGIT

string          = DQUOTE *char DQUOTE
char            = unescaped / escape
unescaped       = %x20-21 / %x23-5B / %x5D-FF
escape          = %x5C ( DQUOTE / %x5C / "/" / "b" / "f" / "n" / "r" / "t"
     )
```

1.2. Descripción general de la gramática

La gramática anterior define tres grandes tipos de mensajes dentro del protocolo: **peticiones**, **respuestas** y **notificaciones**. Cada uno de estos elementos representa una parte esencial del intercambio de información entre cliente y servidor.

1.2.1. Peticiones

Las peticiones son enviadas por el cliente al servidor para solicitar información o gestionar suscripciones. Se contemplan cuatro operaciones principales:

- **LIST**: solicita el listado de ciudades disponibles.

- **GET**: solicita la información meteorológica de una ciudad concreta.
- **SUB**: inicia una suscripción para recibir notificaciones de cambios en una ciudad y, opcionalmente, solo de determinadas variables.
- **UNSUB**: cancela una suscripción previamente realizada.

1.2.2. Respuestas

Las respuestas son generadas por el servidor como contestación a una petición del cliente. Estas pueden contener:

- Confirmaciones satisfactorias con código de estado `200`.
- Datos meteorológicos actuales.
- Listados de ciudades.
- Mensajes descriptivos de éxito o error.
- Errores de protocolo o de procesamiento mediante códigos `400`, `404` y `500`.

1.2.3. Notificaciones

Las notificaciones son mensajes enviados por el servidor cuando existe una suscripción activa y se produce un cambio relevante en los datos meteorológicos. Estas notificaciones incluyen el tipo de mensaje (`NOTIF`), la ciudad afectada y los datos modificados.

1.3. Elementos internos del protocolo

La especificación también define estructuras auxiliares necesarias para representar correctamente la información:

- **city-list**: lista de ciudades disponibles.
- **variable-list**: lista de variables meteorológicas suscritas.
- **weather-data**: conjunto completo de datos meteorológicos de una ciudad.
- **notif-data**: conjunto parcial de variables que han cambiado y que deben notificarse al cliente.

Las variables meteorológicas contempladas en el protocolo son las siguientes:

- `temp`: temperatura.
- `hum`: humedad.
- `pres`: presión.
- `wind`: velocidad del viento.

1.4. Campos y valores literales

Para garantizar coherencia entre implementaciones, la gramática fija explícitamente los nombres de los campos JSON utilizados en cada mensaje. Entre ellos destacan `çommand"`, `çity"`, `"variables"`, `"status"`, `"type"`, `"data"`, `"msg"` y `çurrent"`.

Asimismo, se definen los principales valores literales del protocolo, tales como `LIST`, `GET`, `SUB`, `UNSUB`, `RESP_LIST` y `NOTIF`.

También se establecen los códigos de estado principales:

- `200`: operación correcta.
- `400`: petición incorrecta.
- `404`: recurso no encontrado.
- `500`: error interno del servidor.

1.5. Tipos básicos

Finalmente, la gramática incorpora los tipos básicos que permiten construir el resto de mensajes:

- **string**: cadenas de texto delimitadas por comillas dobles.
- **number**: valores numéricos enteros o decimales, positivos o negativos.
- **char**: caracteres permitidos en cadenas, incluyendo secuencias de escape compatibles con JSON.

Esto asegura que la especificación sea consistente con el formato JSON utilizado por el protocolo y permita validar correctamente la estructura de cada mensaje intercambiado.

1.6. Observación final

Esta especificación ABNF proporciona una definición formal y precisa del protocolo, lo que facilita tanto su implementación como su validación. Además, al establecer una estructura clara para peticiones, respuestas y notificaciones, se mejora la interoperabilidad entre cliente y servidor y se reduce la ambigüedad en el intercambio de mensajes.